Developing a ReAct inspired Agent from scratch for utilizing on Large Language Models (LLMs)

ABZHANOVA ZHANIYA

Degree program code: UA 033 526
Degree program: Business Informatics
Supervisor: Dipl.-Ing. Dr.techn. Marian Lux

Vienna, 2026

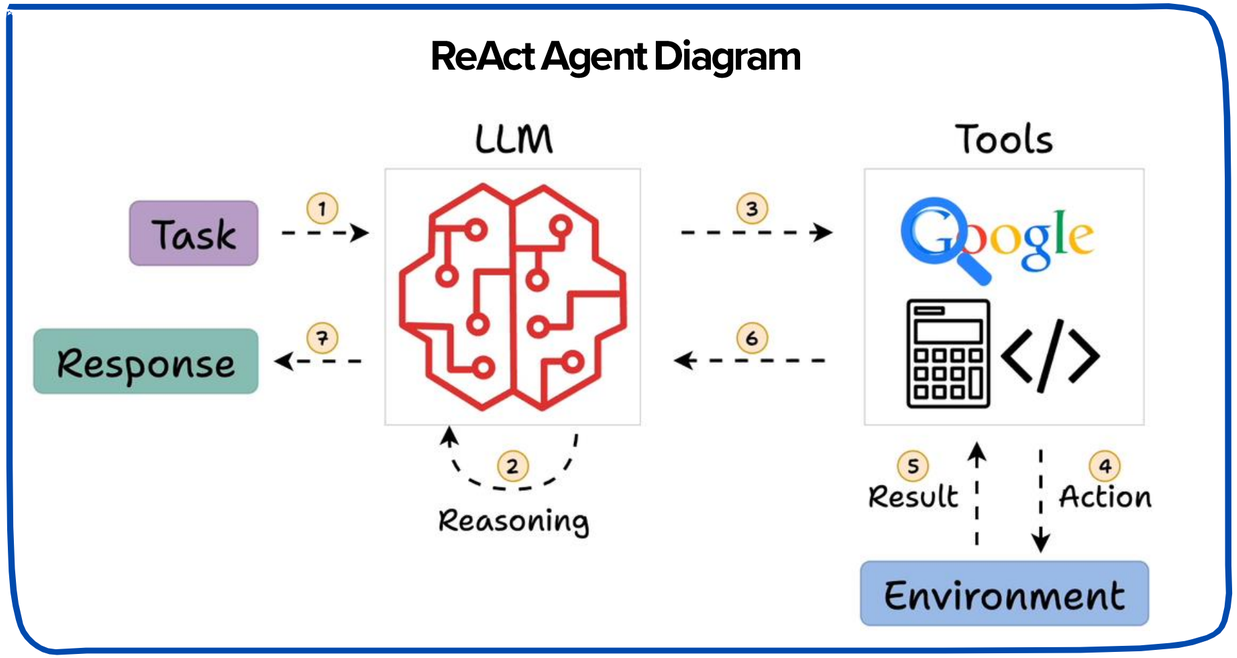
universität wien

1. Motivation

1.1 Definitions

A **Large Language Model (LLM)-** a neural network trained on a very large text dataset to predict the next token (e.g., the next word fragment). Strong capabilities of modern LLMs: analyse vast amounts of information, process it, understand context, and generate answers [1].

ReAct (Reasoning + Acting)- an approach introduced in 2023 that combines an LLM's reasoning ability with its action ability. In ReAct, the model does not only "think" about the problem, but also selects and performs actions during the solving process [2].



Source: https://blog.dailydoseofds.com/p/intro-to-react-reasoning-and-action

1.2 Related Work



LlamaIndex



LangChain





1.3 Practical Challenges in Existing Frameworks

- Too many architectural options [3]
- Tool calling is fragile [4]
- Debugging is difficult [5]

1.4 Goals



➜ make **the control loop** more:

- transparent
- testable
- measurable



➜ **the proposed architecture** improves:

- reliability
- usability
- extendibility

specifically in the constrained setting of local open-source models (with maximum 8B datasets).

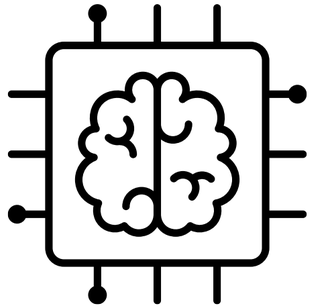
universität wien

2. Design

Major Design Ideas

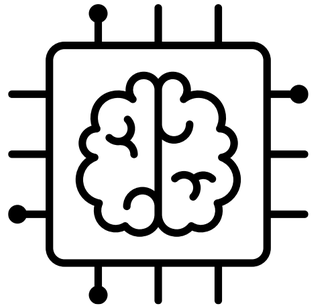
universität wien

1. Pluggable LLM Roles



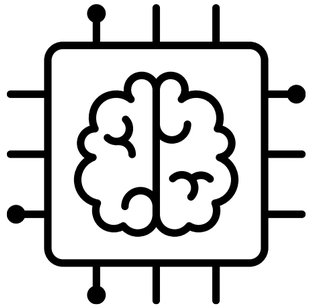
Planner

(decisions)



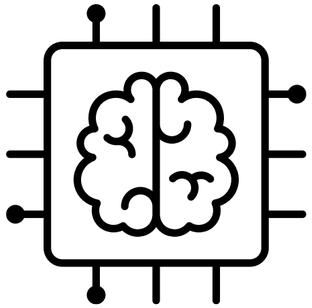
Summariser

(step summaries + final answer)



Refiner

(tool input formatting)



Confidence Model

(scoring step results)

Major Design Ideas

universität wien

2. Customisable Action Catalogue

- USE TOOL
- ANSWER BY ITSELF
- CLARIFY
- RESOLVE PRONOUNS AND OMITTED ENTITIES
- STOP

Major Design Ideas

universität wien

3. Action–Handler Separation



Action

what to do



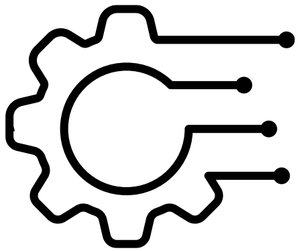
Handler

how it is executed

Major Design Ideas

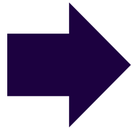
universität wien

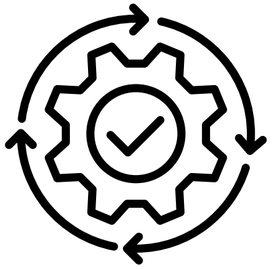
4. Tool Input Refinement



Tool input refinement

according to a tool-specific format

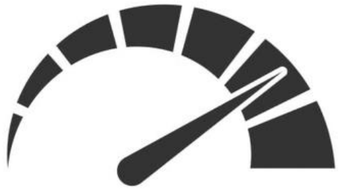




Tool execution

universität wien

5. Customisable Confidence Assessment



The confidence layer is modular

the agent uses multiple small ParameterAssessors, where each assessor checks one quality aspect

universität wien

3. Implementation

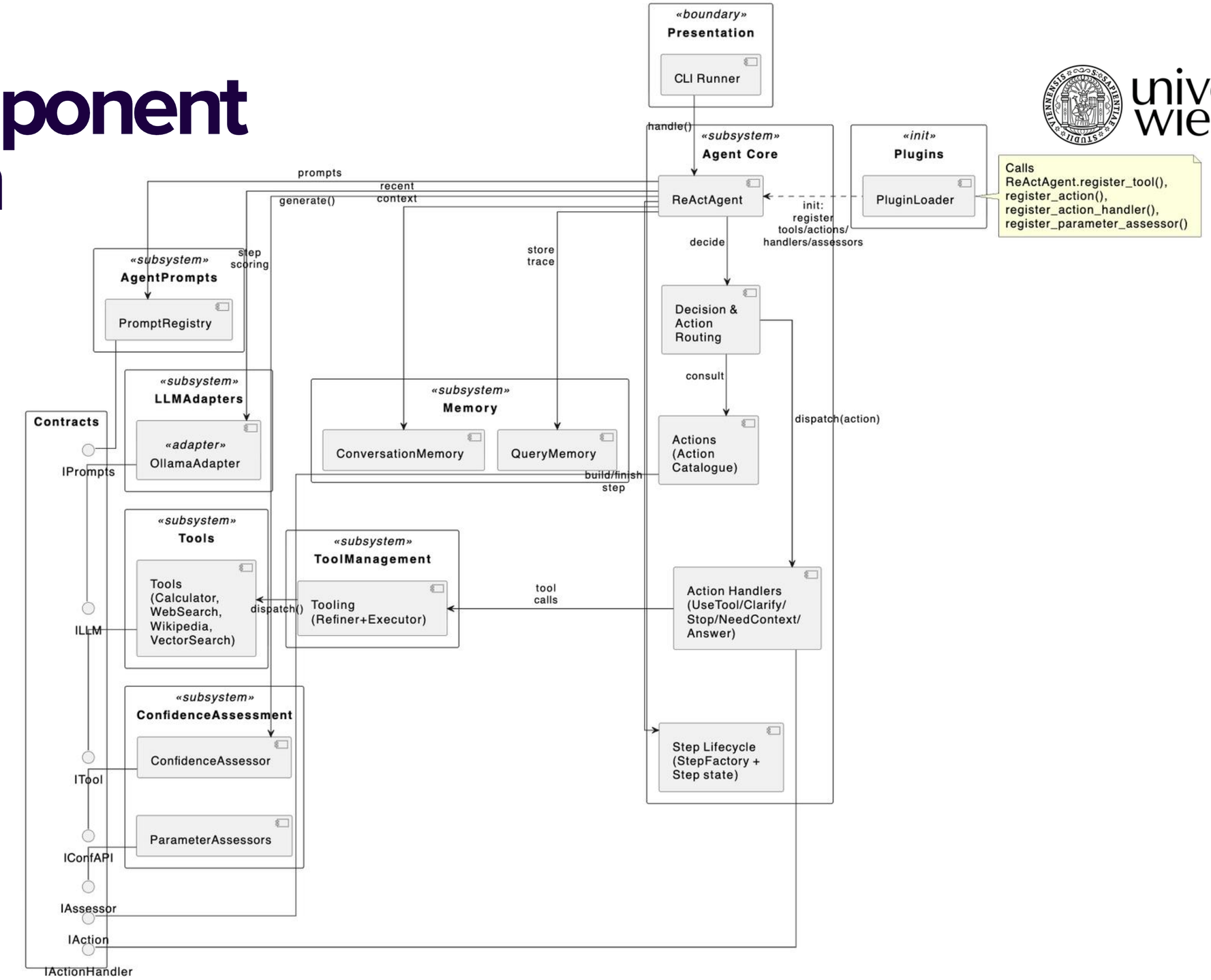
3.1 Main Agent Loop

universität wien

```
(1) Start query session; set status RUNNING; reset per-run state
(2) transcript ← [user question]
(3) if ShouldSeparateQuestion(q) then
    (4) plans ← PlanQuestionSteps(q)
else
    (5) plans ← [single-step plan from q]
end
(6) Save plans to QueryMemory
(7) executed_trace ← [ ]
(8) foreach step in plan up to max_steps do
    (9) decision ← DecideActionForStep(q, plan)
    (10) retries ← 0
    (11) while true do
        (12) Run action handler for decision.action and get step_result
        (13) if decision.action = NEED_CONTEXT then
            (14) optionally rewrite step instruction
            (15) re-decide action for the same step
            (16) continue
        end
        (17) Build step summary evidence
        (18) if step should be confidence-assessed then
            (19) compute confidence conf
        end
        (20) if conf < threshold and retries < max_step_retries then
            (21) decision ← low-confidence recovery decision
            (22) retries ← retries + 1
            (23) continue
        end
        (24) if conf < threshold and retries exhausted then
            (25) store failure message for this step
        end
        else
            (26) persist step trace/evidence to memory
            (27) append successful step to executed_trace
        end
        (28) if step_result.should_stop then
            (29) finalise memory with current answer
            (30) return transcript
        end
        (31) break
    end
end
(32) final ← SynthesiseFinalAnswer(q, executed_trace, context)
(33) append final to transcript; persist transcript and memory
(34) return transcript
```

Algorithm 1: ReActAgent flow from question to final answer

3.2 Component Diagram



universität wien

3.3 Practical Extension Pattern

Add a new tool

universität wien

```python
from datetime import datetime, timezone
from typing import Optional
from my_react_agent.agent_memory.data_structures import Evidence
from my_react_agent.tool_management.tools.agent_tool import AgentTool

class MultiplicationTool(AgentTool):
    @property
    def name(self) -> str: return "multiplier"
    @property
    def description(self) -> str: return "Multiply two numbers: '<a>*<b>'."
    @property
    def refiner_instructions(self) -> str: return "Output ONLY '<a>*<b>'."
    @property
    def refiner_input_format(self) -> str: return "<number>*<number>"
    @property
    def refiner_input_regex(self) -> Optional[str]:
        return r"^\s*-?\d+(\.\d+)?\s*\*\s*-?\d+(\.\d+)?\s*$"

    def execute(self, input: str) -> Evidence:
        a, b = input.split("*", 1)
        result = float(a) * float(b)
        return Evidence(
            tool=self.name,
            content=f"{a.strip()} * {b.strip()} = {result}",
            extracted={"expression": input, "result": result},
            as_of=datetime.now(timezone.utc),
            confidence=0.95,
        )
```

Listing 3.1: Custom AgentTool: MultiplicationTool

```python
from my_react_agent.agent_heart.react_agent import ReActAgent
agent = ReActAgent.create(llm="llama3:8b", tools=[MultiplicationTool()])
print(agent.handle("What is 7*8?"))
```

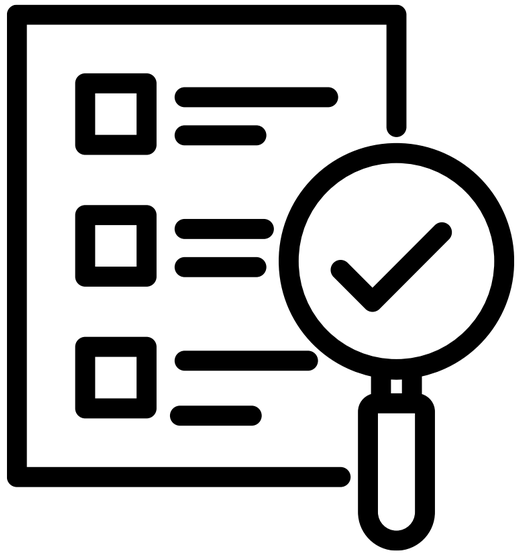
Listing 3.2: Minimal registration

universität wien

Live Demo

4. Assessment

4.1 Assessment Set Up



- **Compared**: My agent vs LangChain vs LlamaIndex
- **Same setup**: qwen3:8b, same tools (web_search, wikipedia, vector_search, calculator)
- **Benchmark**: 5 query types × 3 repeats → 15 runs/agent, 45 runs total
- **Query types**:

1. EUR→USD (time-sensitive)
2. Austria states (fact)
3. Vienna pop + % vs Germany (multi-step)
4. No-context passage (safe handling)
5. Brazil pop with mocked wrong tool output

4.2 Assessment Results

universität wien

Table 4.1: Correctness by query type (successful runs out of 3)

Query type	My Agent	LangChain	LlamaIndex
EUR→USD (adequacy)	3/3	1/3	3/3
Austria states	3/3	3/3	3/3
Vienna + % vs Germany	2/3	1/3	3/3
No-context passage	3/3	0/3	1/3
Brazil population (mock fault)	3/3	0/3	1/3
Overall	**14/15 (93.3%)**	**5/15 (33.3%)**	**11/15 (73.3%)**

Table 4.3: Robustness-focused outcomes

Metric	My Agent	LangChain	LlamaIndex
Mock-value recovery (Brazil)	3/3	0/3	1/3
Safe handling of no-context input	3/3	0/3	1/3

Table 4.2: Efficiency summary over all 15 runs per agent

Agent	Mean latency	Mean iterations	Mean tool calls
My Agent	140.24 s (02:20.24)	2.00	1.73
LangChain	82.29 s (01:22.29)	1.20	1.13
LlamaIndex	45.62 s (00:45.62)	1.27	1.07

universität wien

5. Conclusion

5.1 Conclusion

universität wien

➔ make **the control loop** more:

- transparent ✓ *(explicit step loop + action/handler logic + runtime logs + QueryMemory traces)*
- testable ✓ *(modular subsystems + stable contracts/interfaces + isolated tools/actions/assessors)*
- measurable ✓ *(structured Evidence objects + confidence scores + stored traces + benchmark tables)*

➔ **the proposed architecture** improves:

- reliability ✓ *(tool input refinement + confidence-gated retries + custom ParameterAssessors)*
- usability ✓ *(clear default actions + CLARIFY behavior + central PromptRegistry + plugin-based setup)*
- extendibility ✓ *pluggable tools/LLM adapters/assessors/prompts + (Action–Handler separation)*

specifically in the constrained setting of local open-source models (with maximum 8B datasets).

5.2 Discoveries during Development



1. Schema constraints were more reliable than natural-language instructions

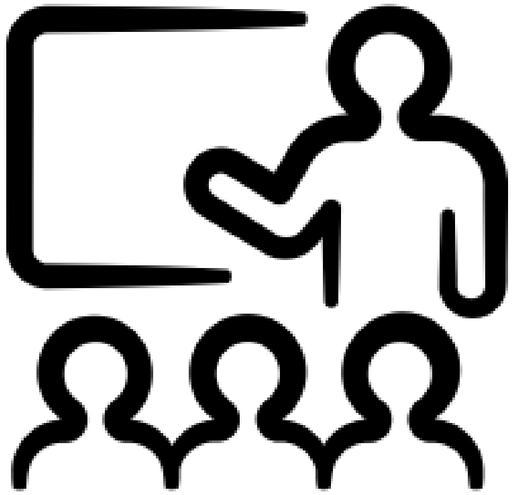


2. Action naming affected routing more than action descriptions



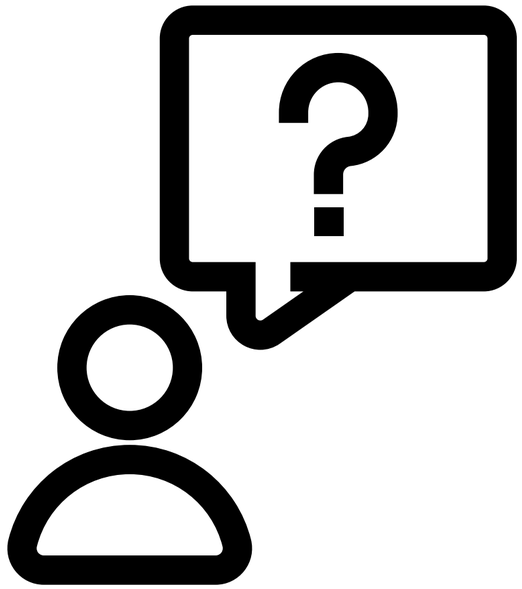
3. Small models required intermediate compression of tool outputs

universität wien



Thank you for your attention!

universität wien



Questions

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal et al., “Language models are few-shot learners,” arXiv:2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” arXiv:2210.03629, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[3] D. Chopra, Y. Lu, T. Turakhia, R. Giordano, N. Ryder, and H. Zuegner, “Measuring agents in production,” arXiv:2512.04123, 2025. [Online]. Available: https://arxiv.org/abs/2512.04123

[4] R. Pedro, D. Castro, N. Santos et al., “From prompt injections to sql injection attacks: How protected is your llm-integrated web application?” arXiv:2308.01990, 2023. [Online]. Available: https://arxiv.org/abs/2308.01990

References

[5] K. Ning, J. Chen, J. Zhang, W. Li, Z. Wang, Y. Feng, W. Zhang, and Z. Zheng, “Defining and detecting the defects of the large language model-based autonomous agents,” arXiv:2412.18371, 2024. [Online]. Available: https://arxiv.org/pdf/2412.18371